

计算机硬件从零到整机

从电、二极管、逻辑门和时钟到最小 CPU 与整机硬件

CPU Books

本书沿着器件、电平、逻辑、状态、时钟、算术、数据通路、控制器、最小 CPU、主存、总线和 I/O 的顺序，把一台计算机从最小可理解部件推导出来。

前言

这本书从硬件本身开始。第一步不是急着给出一个大盒子，而是先把底层问题讲清楚：一根线上连续变化的电压怎样被稳定解释成 0 和 1；一个器件怎样让电流更容易沿某个方向通过；一个受控开关怎样让一个信号决定另一条电流路径是否导通。

沿着这条线继续向前，开关会组成逻辑门，逻辑门会组成加法器、比较器、选择器和译码器。组合逻辑能根据当前输入给出输出，却不能保存过去。为了让硬件一步一步推进，必须引入反馈、锁存器、触发器、时钟、复位、寄存器和状态机。

等状态边界和运算部件都讲清楚后，再进入 ALU、数据通路和控制器。最小 CPU 不在开头出现，而是在寄存器、ALU、数据通路和控制器都已经自然形成之后才出现。这样读者看到 CPU 时，它不是一个突然降临的大盒子，而是一组已经理解过的硬件部件按时钟边界组合起来。

最后再把这个最小计算核心接向主存、总线、设备控制器、中断控制器和启动硬件。读完整本书后，读者能把一台机器从电平、开关、门电路、组合逻辑、时序逻辑、运算部件、数据通路、控制器一路连到整机，而不是只记住一串名词。

本书采用接近 CSAPP (Computer Systems: A Programmer's Perspective) 等经典系统教材的读法：每讲一个抽象，都同时给出机器层证据。读逻辑门时看真值表，也看开关路径；读组合逻辑时看函数，也看延迟和毛刺；读寄存器时看 bit，也看采样边界；读 CPU 时看指令，也看控制信号和数据通路 trace。章节末尾的“经典教材标注”不是额外习题，而是提醒读者用证据检查自己是否真的理解了硬件。

目录

第一部分 器件、开关与组合逻辑	1
第 1 章 电、二极管与受控开关	3
一、先把一根线说清楚	3
二、把连续电压压成稳定电平	4
三、输出必须能驱动下一级	4
四、二极管先提供方向性	5
五、受控开关让信号控制信号	5
第 2 章 从开关到逻辑门	7
一、真值表先把行为列完整	7
二、把真值表翻译成开关路径	7
三、NAND 和 NOR 更接近基础积木	8
四、XOR 表示两个输入是否不同	9
五、门不是理想数学函数	9
第 3 章 组合逻辑电路	11
一、从真值表到门网络	11
二、半加器从两个 bit 相加开始	11
三、全加器把进位接进来	12
四、选择器和译码器负责选路	12
五、组合逻辑没有记忆，也会有短暂毛刺	13
第二部分 状态、时钟与同步电路	14
第 4 章 反馈、锁存器与触发器	16
一、反馈为什么能保存一个 bit	16
二、锁存器：允许时跟随，禁止时保持	16
三、触发器把写入压到时钟边界	16
四、亚稳态来自真实采样边界	17
五、状态边界把计算切成步骤	17
第 5 章 时钟、复位与同步边界	19
一、时钟周期把计算和保存分开	19
二、建立时间和保持时间是采样窗口	19
三、关键路径决定最高可用频率	20
四、复位让状态从已知点开始	20
五、同步设计的基本纪律	20
第 6 章 寄存器、计数器与状态机	22
一、寄存器保存一组 bit	22
二、计数器是带反馈的寄存器	22
三、移位寄存器移动 bit	22
四、状态机把阶段保存下来	23
五、状态机为控制器铺路	23

第三部分 运算部件与数据通路	25
第 7 章 数的表示与算术电路	27
一、无符号数就是按权重求和	27
二、补码让负数也能走加法器	27
三、同一组 bit 会有不同溢出解释	28
四、移位器也是算术部件	28
五、比较可以来自减法和标志位	28
第 8 章 ALU 与标志位	30
一、ALU 的输入输出先从需求看	30
二、减法复用加法器	30
三、结果选择来自多路选择器	31
四、标志位是结果证据	31
五、ALU 是组合部件，保存要靠寄存器	31
第 9 章 数据通路	33
一、寄存器阵列提供多个状态位置	33
二、ALU 输入前必须选路	33
三、写回路径决定结果保存到哪里	33
四、内部连线不能被多个源同时强驱动	34
五、数据通路形成可计算闭环	34
第 10 章 控制器	35
一、控制信号就是电线值	35
二、硬连线控制适合简单规则	35
三、状态机控制多周期动作	36
四、微操作把动作拆成控制信号组	36
五、控制器走向最小 CPU	37
第四部分 最小 CPU 与整机硬件	38
第 11 章 最小 CPU	40
一、为什么还需要动作编码	40
二、PC 保存下一条编码的位置	40
三、取指、译码、执行、写回是一条硬件路径	41
四、分支只是选择下一 PC	41
五、最小 CPU 的边界	42
第 12 章 内存、总线、I/O 与整机	43
一、主存扩展状态空间	43
二、快慢差距逼出层级存储	43
三、总线和互连连接多个目标	44
四、设备通过寄存器、DMA 和中断接入	45
五、复位启动给整机第一步	46

器件、开关与组合逻辑

本部分导读

这一部分只回答一个问题：怎样从可控电流路径得到稳定的 0 和 1，再把多个 0/1 组合成可计算的逻辑网络。讨论对象是器件、电平、门电路和组合逻辑。

电、二极管与受控开关

先看一个最小问题：怎样让一根线可靠表达“是”或“不是”。这根线不是纸上的符号，它有电压、电流、寄生电容、导线阻抗、输入负载和外部噪声。若把 2.5000V 当成 1，把 2.4999V 当成 0，那么一点电源抖动、一点温度变化、一点相邻导线干扰，都可能让后级电路把同一个信号看成两个不同结果。

数字硬件的第一步，不是先背 AND、OR、NOT，而是把连续的物理世界压成可靠的两个状态。后面所有复杂部件都建立在同一个事实上：一根线在某个时刻处于一个可被接收端识别的电平区间。

核心思想

数字硬件的第一个抽象是电平。低电平和高电平不是两个数学点，而是两个带余量的物理区间。

一、先把一根线说清楚

电路里说“一根线是 1”，完整含义不是“线上写着数字 1”，而是“这个节点相对于参考地的电压落在接收端承认的高电平区间内”。这里第一次出现三个词：节点、参考地和电压。

节点可以先理解成多条金属连接汇合成的同一个电气位置。只要这些金属连接得足够好，它们在同一时刻近似共享同一个电压。参考地是整块电路共同约定的 0V 参考点。电压不是孤立存在的绝对高度，而是两个点之间的差值；数字电路里通常说某根信号线是 0V、3.3V 或 5V，实际都是在说它相对于地的电压。

电流则说明电荷正在沿通路流动。若输出节点被一条通路接向电源，它可能被拉到高电平；若被一条通路接向地，它可能被拉到低电平。若两条通路都断开，节点不会立刻变成某个理想值，而可能暂时停在旧电压附近，再被漏电流、干扰或后级输入慢慢推走。若上拉和下拉两条强通路同时打开，电源到地之间形成冲突电流，电平不可靠，还会浪费功耗。

用一个最小按钮电路看这些词。假设一个按钮按下时想表示“请求发生”。天真的想法是把按钮一端接到信号线，另一端接到电源：

```
button pressed -> signal connected to VCC
button released -> signal connected to nothing
```

按下时，信号线被电源路径拉高，这一半没问题。松开时，信号线并没有被明确拉低，它只是断开了。这个节点可能保留刚才的电荷，也可能被旁边导线耦合出一点电压，还可能被后级输入漏电流拖向某个不确定位置。于是接收端有时看到 1，有时看到 0，有时看到过渡电压。这不是程序 bug，而是电路没有给节点一个明确归宿。

正确的第一层想法是：任何要被后级判断的输出节点，都必须在每一种输入情况下被明确地拉向高电平或低电平。按钮松开时需要一条弱下拉路径把信号线拉向地；按钮按下时，按钮路径把它拉向电源。这样，两个物理动作才对应两个稳定电平。

状态	节点实际处境	接收端看到的风险
接向电源	有路径把节点拉高	可以解释为 1
接向地	有路径把节点拉低	可以解释为 0
悬空	没有明确驱动路径	可能保留旧值，也可能被噪声推走

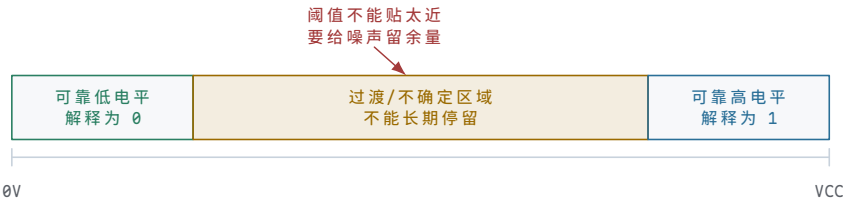
上下同时强驱动

电源和地被短路径连接

电平不可靠，功耗和发热上升

二、把连续电压压成稳定电平

假设一块小电路用 5V 供电。线上的真实电压可能是 0V、0.17V、1.3V、2.6V、4.8V，也可能在切换过程中从 0V 慢慢升到 5V。硬件不能要求每一级都精确读出这个连续数值。更可靠的做法是规定两个稳定区间：低到足够接近地电位时解释为 0，高到足够接近电源电位时解释为 1，中间一段不作为长期稳定状态。



图示：数字电平不是两个精确点，而是两个稳定区间；中间区域留给切换过程和噪声余量。

电压范围	逻辑解释	设计意义
接近地电位	逻辑 0	小噪声不会轻易把它推成 1
中间区域	不作为稳定状态	切换可以经过，但不能长期停留
接近电源电位	逻辑 1	小噪声不会轻易把它拉成 0

这里最重要的不是具体阈值，而是“留余量”。前一级输出的 1 可能不是满满的 VCC，只要高到后一级仍然确信它是 1，就能继续工作。前一级输出的 0 也可能不是绝对 0V，只要低到后一级仍然确信它是 0，就不会误判。数字电路因此能在不完美的物理世界里一级一级传递清楚的信息。

把这个问题放到两级电路之间看更清楚。第一级输出的高电平经过导线、连接器、负载和干扰后可能下降一点；第二级输入阈值如果离高电平区间太近，就会把这个下降误判成 0。噪声余量的意义，就是让“前级还能保证输出的最差 1”和“后级仍然承认为 1 的最低要求”之间留出距离。低电平同理，前级输出的最差 0 也要低于后级仍然承认为 0 的最高要求。

三、输出必须能驱动下一级

一个节点如果只能在空载时看起来像 1，一接上后级就被拖低，它不是可靠数字输出。数字电路中的“输出”必须有驱动能力：它不仅要形成高低电平，还要在连接后级输入、板级走线、封装电容和少量干扰后，仍然把电平维持在接收端承认的范围内。

先看一个失败案例。假设前一级输出通过很弱的路径拉高，单独测量时电压接近 VCC。现在把它接到很多后级输入，每个输入都有一点漏电流和输入电容；切换时还要给这些电容充电。结果可能是：电压上升很慢，甚至还没到高电平可靠区间，接收端就已经在某个时钟边界采样。此时逻辑表达式也许没错，但电路在时间和负载上失败了。

现象	物理原因	后果
高电平被拖低	输出驱动太弱，负载太重	后级可能把 1 判成 0
边沿太慢	节点电容需要充放电时间	采样边界前来不及稳定
多个输出接在一起	两个驱动器给出不同电平	形成冲突电流，结果不可预测
长线受干扰	相邻线切换或外部噪声耦合	电压短暂进入不确定区

这就是为什么多路输出不能简单硬接在一起。某些时刻由甲部件驱动这根线，某些时刻由乙部件驱动这根线，方向切换时必须避免双方同时强驱动。后面讲总线会详细展开，现在只需要先记住：一根线不是抽象共享变量，它同一时刻只能由被允许的硬件驱动。

“扇出”描述一个输出能可靠驱动多少个后级输入。扇出不是数学限制，而是电气限制。后级越多，等效负载越大，边沿越慢，噪声余量越紧。工程上会用缓冲器把一个信号分成多级驱动，或者把长线改成分层网络。这样做不是改变逻辑含义，而是保证物理电平能按时、按强度送到每个接收端。

这个概念也解释了为什么现代芯片外面需要专门的 I/O 单元。芯片内部的逻辑门很小，适合驱动附近的少量门；芯片管脚要面对封装、电路板走线、插槽、电平标准和其他芯片输入，负载大得多。把内部信号直接拿去推外部管脚，通常既不够强，也不够安全。因此真实硬件会把“内部逻辑计算”和“外部管脚驱动”分开设计。

四、二极管先提供方向性

二极管可以先理解成方向性很强的器件。在合适偏置下，它比较容易让电流沿一个方向通过；反向偏置时，它基本阻止电流通过。这个模型不需要一开始展开半导体物理，却已经足够说明硬件设计的一条基本路线：利用器件物理特性，制造可预测的电流路径。

先用一个保护例子理解二极管。假设一个输入节点不希望被外部电压拉到过高或过低，可以用二极管把过大的偏离导向电源或地附近的安全路径。这里二极管做的事不是计算，而是利用方向性限制电流可能走的方向。真实输入保护电路会更复杂，但第一层直觉是相同的：器件不是符号，它通过物理特性约束电流。

再看二极管逻辑。若有两路输入想共同点亮一个指示节点，任意一路为高时都能通过二极管把输出节点拉高，输出就表现出一种 OR 的味道：A 高或 B 高，输出都可能高。

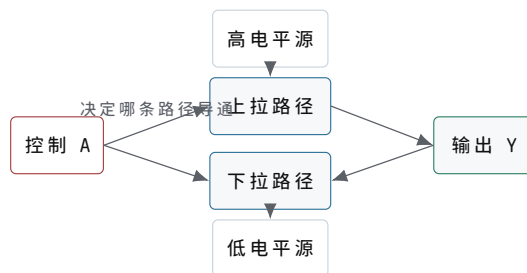
```
if A can drive through a diode:
    Y may be pulled high
if B can drive through a diode:
    Y may be pulled high
```

这个例子有用，但它也暴露出二极管逻辑的局限。第一，二极管会带来压降，输出高电平可能比输入高电平低一截；级联多了，电平余量会越来越紧。第二，二极管只能提供方向性，不能主动把一个偏弱的高电平重新恢复成强高电平。第三，它不方便实现取反，而取反是后面构造通用逻辑必须具备的能力。

取反为什么重要？因为许多条件天然需要反向解释。传感器未触发时允许运行，触发时禁止运行；按钮未按下时保持，按下时改变。若电路只能表达“有一路导通”，却不能把 1 变成 0、把 0 变成 1，那么可组合的逻辑很快就受限。要继续向前，需要一个信号能控制另一条电流路径是否导通。

五、受控开关让信号控制信号

晶体管的细节很深，但作为第一层硬件模型，可以先把它当成受控开关。控制端不是直接给输出供电，而是决定输出节点到某条电源路径之间是否接通。



图示：受控开关的核心不是“输入直接变输出”，而是输入决定输出节点被哪条电流路径驱动。

```
control = 0 -> one current path is open  
control = 1 -> one current path is closed
```

这样就出现了真正的逻辑能力：一个输入信号可以决定输出节点被拉向哪里。若输入为 1 时下拉路径导通，输出节点被拉向低电平；若输入为 0 时上拉路径导通，输出节点被拉向高电平。这个结构就是取反的直觉来源。

把开关串联，可以表达“两个条件都满足才有通路”；把开关并联，可以表达“任意一个条件满足就有通路”。这里还没有正式讲 AND、OR、NOT，但它们的硬件根已经出现了：逻辑关系不是凭空的符号表，而是受控电流路径的组合关系。

本章得到的不是完整计算机，而是几个底层积木：参考地、节点、电压、电流、电平区间、噪声余量、输出驱动能力、方向性器件、受控开关、上拉路径和下拉路径。它们回答了“0 和 1 怎样落到物理世界里”这个问题。

下一章开始把这些积木连成逻辑门。到那时，NOT、AND、OR、NAND、NOR 和 XOR 不再只是表格中的符号，而是具体的开关网络：哪些路径导通，哪些路径断开，输出节点为什么会被拉成 0 或 1。

本章的机器证据不是“会背电压、电流、二极管、晶体管”这些词，而是能检查一根线在每种情况下是否有明确驱动路径。

- 纸上实验：给一个按钮输入，分别画出按下、松开、抖动、断线四种情况，标出信号线由谁拉高、由谁拉低、何时悬空。
- 位级证据：任何输出节点都要能写成“高电平路径是否导通、低电平路径是否导通、是否存在冲突电流、是否存在悬空”四项。
- 时间证据：输出从 0 变 1 不是瞬时动作，负载电容越大，进入可靠高电平区间越晚。
- 检查题：若一个节点单独测量是 1，但接上四个后级后有时被读成 0，优先怀疑逻辑表达式，还是驱动能力和噪声余量？

经典系统教材常把“抽象”和“实现”放在同一页上看。这里的抽象是 bit，实现是一段电压区间和受控电流路径。后面所有逻辑门、寄存器、总线和 I/O 都不能违反这个底层事实。

从开关到逻辑门

上一章只得到一件事：输入信号可以控制一条电流路径是否导通。现在把这个能力用起来，让多个输入共同决定一个输出。先不要急着背 AND、OR、NOT 的符号，先看两个很小的硬件需求。

第一个需求是安全允许灯。小设备只有在“保护盖合上”和“急停未按下”两个条件同时成立时，允许灯才亮。第二个需求是故障灯。只要温度报警或电流报警任意一个出现，故障灯就要亮。第三个需求是反向控制：某个按钮按下时，输出反而要变低。这三个需求分别逼出 AND、OR、NOT。

一、真值表先把行为列完整

门电路的第一步不是画线，而是把输入组合列完整。两个输入 A、B 只有四种组合。电路必须对每一种组合给出明确输出，而且每一行都要满足上一章的要求：输出节点不能悬空，不能上下冲突。

先看安全允许灯。A 表示保护盖合上，B 表示急停未按下。允许灯亮的条件很严格：少一个条件都不行。

A	B	允许灯	为什么
0	0	0	盖子没合上，急停也不是可运行状态
0	1	0	急停条件满足，但盖子没合上
1	0	0	盖子合上了，但急停条件不满足
1	1	1	两个安全条件同时满足

这张表就是 AND 的行为。它不是抽象数学定义，而是“两个条件都满足才允许”的硬件约束。若某个实现让 A=1、B=0 时灯也亮，那就不是 AND 近似得不够好，而是安全行为错了。

再看故障灯。A 表示温度报警，B 表示电流报警。故障灯的规则反过来：只要有一个坏消息出现，就要亮。

A	B	故障灯	为什么
0	0	0	没有任何故障证据
0	1	1	电流报警已经足够触发故障
1	0	1	温度报警已经足够触发故障
1	1	1	两个故障同时存在，当然要报警

这张表就是 OR 的行为。AND 和 OR 的差别不在符号长什么样，而在需求不同：一个要求所有条件同时成立，一个要求任意证据成立。

NOT 只有一个输入，但同样要列完整。若 A 表示“禁止信号”，输出 Y 表示“允许动作”，那么 A=0 时 Y=1，A=1 时 Y=0。它解决的是反向解释：输入出现时，输出反而关闭。

二、把真值表翻译成开关路径

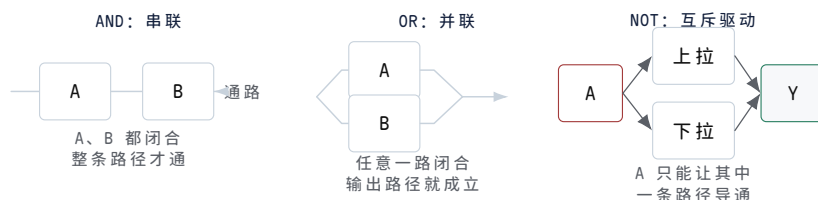
真值表说清楚“应该怎样”，开关路径说明“为什么会这样”。用受控开关看 AND，最自然的结构是串联。两个开关串在同一条路径上，只有 A 和 B 都让自己的开关闭合时，整条路径才通。任意一个开关断开，路径就断开。

把四种输入逐行走一遍，AND 就不需要死记：

A	B	串联路径状态	输出为什么是这个结果
0	0	两个开关都断开	没有完整导通路径，输出不能被拉成 1
0	1	A 断开，B 闭合	串联路径仍被 A 切断
1	0	A 闭合，B 断开	串联路径仍被 B 切断
1	1	两个开关都闭合	从源到输出的路径完整，输出成立

OR 更像并联。A 控制一条路径，B 控制另一条路径。只要任意一路闭合，输出就能被拉向目标电平。只有两路都断开时，输出才没有这类导通路径。

NOT 的核心是互斥驱动。输入为 0 时，上拉路径导通、下拉路径断开，输出被拉成 1；输入为 1 时，上拉路径断开、下拉路径导通，输出被拉成 0。这里要特别注意：不是把一个输入“改名”为相反值，而是用这个输入选择输出节点接向高电平路径还是低电平路径。



图示：串联、并联、互斥上拉/下拉分别给出 AND、OR、NOT 的开关直觉。

这张图只表达直觉，还不是完整晶体管级电路。真实门要同时安排上拉网络和下拉网络，保证每一种输入组合下输出都被明确拉高或拉低。只画“能让输出为 1 的路径”还不够，还必须说明输出为 0 时由谁负责拉低。

三、NAND 和 NOR 更接近基础积木

如果只从逻辑式看，AND 和 OR 很自然；但在开关网络里，NAND 和 NOR 往往更像基础积木。原因是它们天然带反相输出，更容易形成互补的上拉和下拉结构：当下拉网络导通时输出被拉低；当下拉网络不导通时，上拉网络负责把输出拉高。

以 NAND 为例。NAND 的输出只有在 $A=1$ 且 $B=1$ 时为 0，其余三种情况为 1。

A	B	NAND 输出	开关直觉
0	0	1	下拉串联路径断开，上拉负责拉高
0	1	1	A 断开，下拉路径不完整
1	0	1	B 断开，下拉路径不完整
1	1	0	A、B 都闭合，下拉路径完整，输出被拉低

这就是为什么“输出那一端倒着看”很重要。问一个门是否可靠，不只问逻辑表达式对不对，还要问每一行输入下：谁拉高，谁拉低，会不会悬空，会不会上下同时强驱动。NAND 好用，是因为它能用清楚的下拉条件和对应上拉条件覆盖所有输入行。

只用 NAND 就能构造 NOT、AND、OR：

```

NOT A = A NAND A
A AND B = (A NAND B) NAND (A NAND B)
A OR B = (A NAND A) NAND (B NAND B)

```

这几行说明，硬件不需要为每种逻辑关系重新发明一种器件。只要有一类稳定、可复制、驱动能力可控的基础门，就能组合出更复杂的逻辑网络。真实芯片里会有

多种标准单元，例如不同驱动强度的 NAND、NOR、反相器、缓冲器、选择器，但“复杂逻辑来自小门组合”这条线没有变。

四、XOR 表示两个输入是否不同

XOR 的输出在两个输入不同的时候为 1，相同的时候为 0。它一开始看起来比 AND、OR 难，因为它不是简单的串联或并联；但它解决的是一个非常具体的问题：检测两个 bit 是否不同。

A	B	A XOR B	解释
0	0	0	两个输入相同
0	1	1	两个输入不同
1	0	1	两个输入不同
1	1	0	两个输入相同

可以把 XOR 拆成两种会输出 1 的情况：A=1 且 B=0，或者 A=0 且 B=1。写成逻辑表达式就是：

$$A \text{ XOR } B = (A \text{ AND NOT } B) \text{ OR } (\text{NOT } A \text{ AND } B)$$

这条式子不要只当代数记忆。它对应两条证据路径：第一条证明“只有 A 为 1”，第二条证明“只有 B 为 1”。最后的 OR 合并这两条证据。若两个都为 0，没有证据成立；若两个都为 1，两条“只有一个为 1”的证据也都不成立。

XOR 后面会反复出现。两个 bit 相加时，和位就是“两个输入是否不同”；比较两个 bit 时，XOR 可以告诉我们它们是否不同；再接一个 NOT，就能得到“两个输入是否相同”。所以 XOR 虽然比 AND、OR 复杂，但它不是花哨门，而是算术和比较的基础小部件。

五、门不是理想数学函数

门电路在纸上像瞬间给出输出的函数，真实硬件却至少有三个限制。第一，输入变化后，输出需要传播延迟才稳定。第二，一个输出能驱动的后级输入数量有限，后级太多会让切换变慢，甚至让电平余量变差。第三，门的输出必须重新形成清楚的 0/1，不能只靠一个偏弱路径勉强改变节点电压。

限制	含义	后续影响
传播延迟	输出不会在输入变化的同一瞬间稳定	组合逻辑层数越多，稳定越慢
扇出	一个输出只能可靠驱动有限后级	大网络需要缓冲器和分层布线
电平恢复	输出要重新形成清楚 0/1	不能只追求逻辑表达式正确

看一个小反例。若把十几个门的输入全接到同一个弱输出上，真值表仍然可以写得很漂亮，但物理电路可能切换很慢。输出从 0 变 1 时，要给所有后级输入电容充电；如果下一级在它还没进入可靠高电平区间时就读取，就可能看到旧值或不确定值。于是后面讲组合逻辑时，不能只数门的个数，还要看最长路径、负载和稳定时间。

本章把第一章的受控开关变成了门。AND 来自串联条件，OR 来自并联证据，NOT 来自互斥上拉/下拉，NAND/NOR 提供更适合复制的基础积木，XOR 提供“是否不同”的判断。下一章会把多个门接成组合逻辑网络。那时每个电路都要同时满足两件事：逻辑上真值表正确，物理上能在规定时间内稳定，并且能继续驱动后级。

本章要用两张表验收同一个门：真值表证明逻辑行为，路径表证明每一行为什么会被拉成对应电平。

- 纸上实验：任选一个二输入门，列出 4 行真值表，再为每一行写出“上拉网络导通/断开、下拉网络导通/断开”。
- 机器证据：合法门在稳定输入下不能让输出悬空，也不能让上拉和下拉强路径同时导通。
- 组合证据：用 NAND 构造 NOT、AND、OR 时，不要只写代数式，还要说明输出是否经过了电平恢复。
- 检查题：一个 XOR 电路真值表正确，但比 AND 慢很多，这个慢来自功能复杂度、门级层数、扇出，还是三者都有可能？

经典教材的写法会反复提醒：逻辑代数不是电路本身。代数式说明目标行为，门网络说明实现路径，延迟和负载说明这条路径能不能在真实机器里按时工作。

第 3 章

组合逻辑电路

组合逻辑的特点很直接：输出只由当前输入决定，不保存过去。它像一个硬件函数，但这个函数不是一步一步执行的过程，而是一张由门和连线组成的网络。输入变化后，信号沿多条路径同时传播；等延迟过去，输出稳定到当前输入对应的值。

一、从真值表到门网络

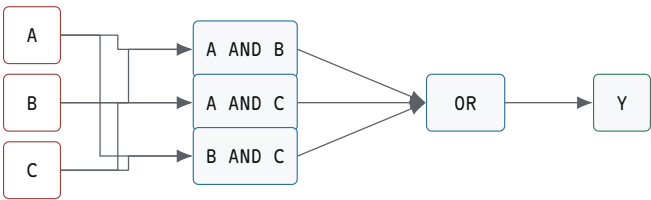
先做一个简单判断器：输入 A、B、C 表示三个传感器，要求至少两个传感器为 1 时输出 1。这个需求不能只凭直觉画线，先列出关键行：

A	B	C	Y	原因
0	0	0	0	一个条件都没有
1	0	0	0	只有一个条件
0	1	0	0	只有一个条件
0	0	1	0	只有一个条件
1	1	0	1	A 和 B 同时满足
1	0	1	1	A 和 C 同时满足
0	1	1	1	B 和 C 同时满足
1	1	1	1	三个都满足

从表中可以直接得到一个门网络：

Y = (A AND B) OR (A AND C) OR (B AND C)

这条表达式里有三条“证据路径”。第一条路径证明 A、B 同时为 1；第二条路径证明 A、C 同时为 1；第三条路径证明 B、C 同时为 1。最后的 OR 不是随便合并，而是在问：三条证据里有没有任意一条成立。这样看，门网络就不只是公式变形，而是把真值表中的每一种输出为 1 的原因接成硬件。



图示：“至少两个为 1”的组合逻辑由三条证据路径和一个合并路径构成。

这个例子展示了组合逻辑的一般做法：先把行为说清楚，再把能让输出为 1 的条件提出来，最后用门电路连接。若以后发现路径太长或门太多，可以再优化，但不能跳过“行为完整”这一步。

二、半加器从两个 bit 相加开始

两个 bit 相加，输出不止一个 bit。因为 1+1 得到二进制 10，需要一个和位 S 和一个进位 C。

A	B	S	C
0	0	0	0
0	1	1	0

1	0	1	0
1	1	0	1

观察表格可以看到，S 在 A、B 不同时为 1，所以：

```
S = A XOR B
C = A AND B
```

这就是半加器。它已经完成了一位相加，但它还缺一个输入：来自低位的进位。只要要做多位加法，最低位之外的每一位都不能只看 A 和 B，还要看低一位是否进上来。

三、全加器把进位接进来

全加器输入为 A、B、Cin，输出为 S、Cout。可以先把 A 和 B 用半加器相加，得到临时和 S1 和进位 C1；再把 S1 和 Cin 用第二个半加器相加，得到最终和 S 和第二个进位 C2；最后把两个进位合并。

```
S1 = A XOR B
C1 = A AND B
S = S1 XOR Cin
C2 = S1 AND Cin
Cout = C1 OR C2
```

把全加器一位一位串起来，就得到串行进位加法器。它的逻辑很清楚：第 0 位算出进位，传给第 1 位；第 1 位再把进位传给第 2 位。问题也很清楚：若最低位产生的进位一路传到最高位，最高位必须等前面所有进位稳定后才能稳定。



最坏情况：进位必须从最低位一路传到最高位，最高位结果最后稳定

图示：串行进位加法器逻辑简单，但最长延迟路径沿进位链传播。

这就是组合逻辑里第一次真正遇到“逻辑正确”和“速度足够”的差别。真值表保证最后结果正确，传播延迟决定结果多久之后才可靠。位宽越大，简单串行进位加法器的最坏路径越长。

四、选择器和译码器负责选路

多路选择器解决“多个输入选一个输出”。2 选 1 选择器的行为是：

```
if sel = 0, Y = A
if sel = 1, Y = B
```

这里的 if 只是描述行为，电路本身没有顺序执行。它的门级表达式是：

```
Y = (NOT sel AND A) OR (sel AND B)
```

选择器的作用很基础。若一条输出线可能来自加法器，也可能来自传感器，也可能来自某个寄存器，那么必须有一个明确的选择信号决定哪一路接到输出。没有选择器，多路来源同时驱动同一条线就会冲突。

译码器做相反方向的事：把一个二进制编号展开成 one-hot 信号。2-bit 输入可以选中 4 条输出中的一条。

输入	输出
00	Y0=1, Y1/Y2/Y3=0
01	Y1=1, 其他为 0

10	Y2=1, 其他为 0
11	Y3=1, 其他为 0

选择器把多路收成一路，译码器把编号展开成多路选择。后面无论是寄存器阵列、数据通路，还是存储器地址选择，都会反复看到这两个结构。

五、组合逻辑没有记忆，也会有短暂毛刺

组合逻辑只看当前输入，不保存过去。若输入 A 从 0 变成 1，多个门路径的延迟不同，输出可能先短暂抖一下，再稳定到最终值。这种短暂错误叫毛刺。毛刺不一定会造成问题，但前提是后级不要在毛刺期间采样。

这就自然引出下一章。只靠组合逻辑，硬件能计算当前输入对应的输出，却不能决定“什么时候把结果当真”，也不能把某个结果保存到下一步。要让硬件分步骤工作，必须引入能保存状态的电路。

组合逻辑的验收标准是“输入稳定后，输出在有限延迟后由当前输入唯一决定”。这句话里有三个关键词：当前输入、有限延迟、唯一决定。

- 纸上实验：给定表达式 $Y=(A \text{ AND } B) \text{ OR } (\text{NOT } A \text{ AND } C)$ ，列真值表，标出最长门级路径，并指出输入变化时可能出现毛刺的路径。
- 位级证据：半加器的 Sum 是 XOR，Carry 是 AND；全加器不是新魔法，而是把进位输入纳入同一张真值表。
- 结构证据：mux 解决多源选一路，decoder 解决编号选一项；寄存器阵列、内存和数据通路后面都会复用这两个结构。
- 检查题：若组合网络输出短暂从 0 跳到 1 又回到 0，而最终值正确，它一定是 bug 吗？答案取决于后级是否在毛刺期间采样。

经典教材常用“抽象位函数 + 实际延迟模型”一起解释组合电路。只看位函数会漏掉毛刺和关键路径；只看延迟又会失去逻辑行为。两者合在一起，才是硬件设计对象。

状态、时钟与同步电路

本部分导读

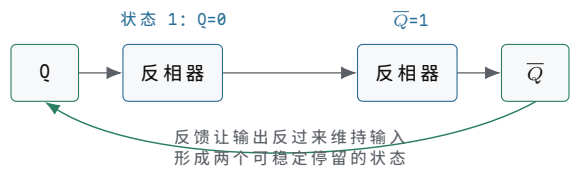
组合逻辑没有记忆。要让硬件一步一步推进，必须引入反馈、锁存器、触发器、时钟、复位、寄存器、计数器和状态机。这一部分只讲状态怎样稳定保存、怎样按边界更新。

反馈、锁存器与触发器

组合逻辑没有记忆。按钮按下时输出为 1，按钮松开后输出回到 0。若硬件需求是“按一下后保持亮”，只用组合逻辑就不够了。要保存过去，电路必须让当前输出影响未来输出，也就是把输出的一部分反馈回输入。

一、反馈为什么能保存一个 bit

两个反相器首尾相接，会形成两个稳定状态。若左边节点为 0，右边经过反相后为 1；右边的 1 再反相回来，正好维持左边为 0。反过来，左边为 1、右边为 0 也能稳定。只要没有外部强制改变，这个状态会一直保持。



图示：两个反相器相互支撑，电路不是记住“变量名”，而是停在一个稳定物理状态上。

这件事说明“保存一个 bit”不是抽象魔法，而是电路中某些节点在反馈作用下停留在一个稳定状态。一个稳定状态解释为 0，另一个稳定状态解释为 1。状态的本质，是电路有能力在输入不再变化时仍然维持某个内部结果。

但单纯反馈还不够。若我们无法可靠地把它从 0 改成 1，或从 1 改成 0，它就只是一个会停住的环，不是可用存储单元。因此下一步要加入写入控制。

二、锁存器：允许时跟随，禁止时保持

锁存器可以理解为带写使能的 1-bit 存储单元。写使能有效时，输入 D 可以影响输出 Q；写使能无效时，反馈通路维持原来的 Q。

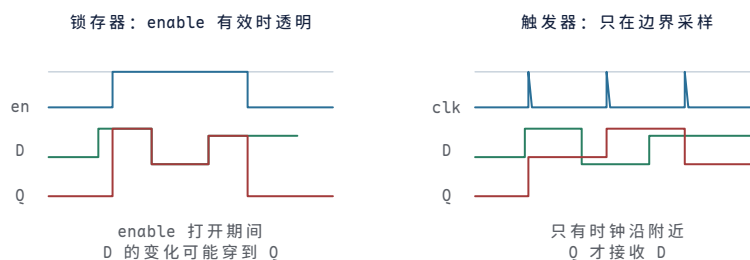
写使能	数据输入	输出行为
有效	0 或 1	输出跟随输入变化
无效	任意	输出保持原值

例如一个“按住才能设置”的电路中，enable 为 1 时，D=1 会把锁存器写成 1；enable 回到 0 后，即使 D 变回 0，Q 仍保持 1。这样，过去某一刻的输入就被硬件保存下来了。

锁存器的问题在于透明窗口。只要 enable 有效，输入变化就可能一路穿透到输出。若多个锁存器共用同一个 enable，并且它们之间还接着组合逻辑，状态可能在一个打开窗口内穿过多级，边界变得模糊。

三、触发器把写入压到时钟边界

触发器解决的是“什么时候写入”的问题。它不像锁存器那样在一段时间内透明，而是在时钟沿附近采样输入，把采样值保存到输出。



图示：锁存器在窗口内跟随输入，触发器把写入压到离散时钟边界。

```
on clock edge:
    Q = D
between clock edges:
    Q holds
```

这让大电路有了清楚节奏。一个时钟沿之后，上一批触发器输出新状态；这些状态经过组合逻辑传播，形成下一批输入；到下一个时钟沿，目标触发器再统一采样。于是“计算”和“保存”被分开了。

可以用一个两步计数需求理解它。第一步，当前值进入加一电路；第二步，下一个时钟沿把加一结果保存。如果没有触发器边界，加一结果又会立刻回到输入，再被加一，再回到输入，电路会在一个周期内不断变化。触发器把反馈切成一拍一拍的更新。

四、亚稳态来自真实采样边界

触发器不是数学上的瞬间拍照。若输入 D 正好在时钟沿附近变化，内部节点可能短时间落在不稳定区域，既不像明确 0，也不像明确 1。这个现象叫亚稳态。

亚稳态不是因为设计者不会写逻辑表达式，而是物理器件在边界条件下真实存在的行为。多数情况下，它会在一小段时间后落回 0 或 1，但落回哪边、需要多久，无法按普通组合逻辑那样精确保证。

这直接带来设计纪律：被触发器采样的输入，必须在时钟沿前后保持稳定。来自另一个时钟节奏或外部世界的信号，不能直接拿来控制大面积电路，通常要先经过同步结构，降低亚稳态扩散的概率。

五、状态边界把计算切成步骤

本章得到三件关键东西。反馈提供两个可稳定停留的状态；锁存器提供“允许写入、否则保持”的能力；触发器把写入限制到时钟边界。

从这里开始，硬件不再只是“输入一变，输出跟着变”的组合网络，而可以分步骤工作。下一章会把这些状态边界组织成全局节奏：时钟周期、复位、建立时间、保持时间和关键路径。

本章第一次出现真正的“机器状态”。状态不是变量名，而是由反馈和存储节点保持住的一组 bit。

- 纸上实验：画出一个 D 触发器前后各接一段组合逻辑，标出当前周期的旧 Q、组合路径上的 D、下一个时钟沿后的新 Q。
- 状态证据：锁存器在使能窗口内跟随输入，触发器只在时钟边界采样；这两个边界不能混用。
- 亚稳态证据：若 D 在采样窗口附近变化，不能用普通真值表预测内部节点何时稳定，只能用同步器降低风险。

- 检查题：一个反馈加一电路如果没有触发器，为什么可能在一个周期内连续变化？触发器到底切断了哪条路径？

经典教材通常会把“组合逻辑”和“状态逻辑”分开建模：组合逻辑计算下一值，状态元件在边界保存下一值。后面 CPU 的 PC、寄存器、状态机和控制器都遵守这个模型。

时钟、复位与同步边界

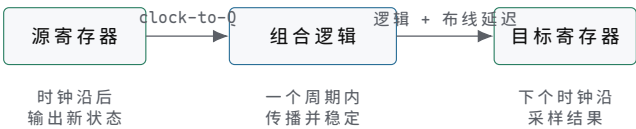
如果每个存储单元随时都能改变，大电路很快会混乱。同步电路采用统一节奏：一个时钟沿让一批状态更新，随后组合逻辑根据这些状态传播，直到下一个时钟沿再保存新结果。时钟不是装饰，也不只是“频率数字”，它是状态更新边界。

一、时钟周期把计算和保存分开

先看一条最基本的寄存器到寄存器路径：

```
source register -> combinational logic -> destination register
```

时钟沿到来后，源寄存器输出旧状态对应的新值。这个值进入组合逻辑，经过门延迟和布线延迟，最终到达目标寄存器输入。下一个时钟沿到来时，目标寄存器采样这个结果。



图示：同步路径的周期预算来自寄存器输出延迟、组合逻辑延迟、布线延迟和目标寄存器建立时间。

```
after clock edge:
  source register output changes
between edges:
  combinational logic settles
at next edge:
  destination register captures
```

因此频率不是越高越好。频率提高意味着周期缩短，留给组合逻辑稳定的时间变少。若周期短到最慢路径来不及稳定，目标寄存器就可能采到旧值、过渡值或错误值。

二、建立时间和保持时间是采样窗口

触发器采样不是数学瞬间。输入 D 必须在时钟沿到来前稳定一小段时间，这叫建立时间；时钟沿之后还要继续稳定一小段时间，这叫保持时间。

约束	含义	违反后果
建立时间	采样前输入要提前稳定	采到错误值或不稳定值
保持时间	采样后输入要继续稳定	新旧值混入采样窗口
传播延迟	上一级输出和组合逻辑需要时间	周期太短会来不及

举一个具体路径。源寄存器在时钟沿后并不会立刻输出新值，它有一个 clock-to-Q 延迟；组合逻辑本身有延迟；线也有延迟。目标寄存器又要求输入在下一个时钟沿前提前稳定。时钟周期必须覆盖这些时间。

```
clock period >= source clock-to-Q
                  + combinational delay
                  + wire delay
```

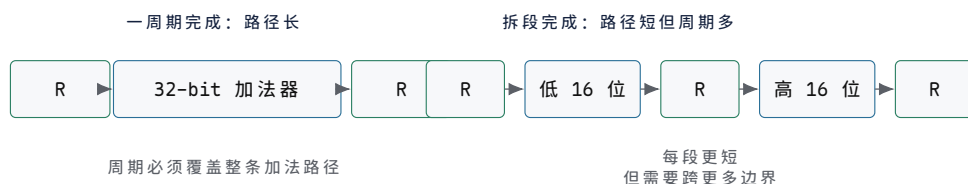
+ destination setup time

保持时间看起来更小，却同样危险。若源寄存器输出太快、路径太短，目标寄存器可能在同一个时钟沿之后立刻看到新值，从而破坏它本应采旧值的窗口。长路径会限制最高频率，短路径也可能破坏保持约束。

三、关键路径决定最高可用频率

一块同步电路里会有很多寄存器到寄存器路径。最长、最慢、最限制周期的那条叫关键路径。它可能是一串加法器，也可能是多级选择器，也可能是远距离布线加上复杂译码。

例如一个 32-bit 串行进位加法器接在两个寄存器之间。最低位进位要一位一位传到最高位，最高位结果最后才稳定。如果把这个加法器放在一个周期里完成，那么周期至少要等最坏进位链走完。若把运算拆成更短的几段，中间插入寄存器，单个周期可以变短，但完成一次完整运算需要更多周期。



图示：关键路径变短可以提高单周期频率，但插入状态边界会改变完成一次动作所需的周期数。

这就是硬件设计常见取舍：更短周期通常要更多状态边界；更少周期通常要求更长组合路径。时钟不是单独设置出来的，它受电路结构约束。

四、复位让状态从已知点开始

触发器通电后的值不一定符合设计预期。若一组状态寄存器随机落在不同值，后面的组合逻辑可能立刻进入非法状态。复位信号的作用，是把关键状态单元带到已知点，例如计数器清零、状态机回到 idle、控制寄存器进入安全默认值。

复位本身也要讲边界。若复位释放时，有的触发器先退出复位，有的后退出复位，状态机可能看到一半新状态、一半旧状态。常见处理是异步拉低或拉高复位以便快速进入安全状态，再同步释放，让所有相关触发器在清楚的时钟边界恢复工作。

复位不是“清零所有东西”这么简单。哪些状态需要复位，哪些数据寄存器可以不复位，哪些输出在复位期间必须安全，都要由硬件需求决定。盲目复位太多节点会增加布线和时序压力；复位太少又可能让电路从未知状态开始。

五、同步设计的基本纪律

到这里，同步设计可以压成三条纪律。第一，状态只在明确边界更新。第二，组合逻辑必须在下一个边界前稳定。第三，跨边界信号必须先处理，不能假设外部变化刚好对齐本地时钟。

下一章会把这些纪律用于更大的状态结构。多个触发器并排形成寄存器；寄存器加反馈形成计数器；寄存器保存阶段编号，组合逻辑决定下一阶段，就形成有限状态机。

同步设计的核心标注是时序账本。每条从寄存器出发、经过组合逻辑、到达另一个寄存器的路径，都要交代传播延迟、建立时间、保持时间和时钟偏斜。

- 纸上实验：给一条路径写出 $T_{clk} \geq t_{CQ} + t_{comb} + t_{setup} + t_{skew}$ ，再解释每一项对应哪个硬件现象。
- 机器证据：关键路径决定最高频率，不是平均路径决定；一条慢路径就能

限制整块同步电路。

- 复位证据：复位要让控制状态进入已知安全点，但释放复位也要有同步边界，不能让状态机半边先跑。
- 检查题：把一个 32-bit 加法器拆成两个 16-bit 阶段后，频率可能提高，但一条加法指令的总延迟一定降低吗？

经典教材强调“时间也是硬件资源”。面积、功能、时钟周期、吞吐和延迟互相牵制。一个逻辑表达式正确但错过采样窗口，在机器里仍然是错误电路。

第 6 章

寄存器、计数器与状态机

一个触发器保存 1 bit。把多个触发器并排，并让它们共享同一个时钟边界，就得到多 bit 寄存器。再把寄存器输出送进组合逻辑，把组合逻辑结果送回寄存器输入，就能让硬件按规则一步一步改变状态。

一、寄存器保存一组 bit

寄存器可以理解为一组共享时钟和写使能的触发器。若写使能有效，在时钟沿采样输入；若写使能无效，保持原值。

```
on clock edge:
    if write_enable:
        Q = D
    else:
        Q holds
```

寄存器的意义不只是“能存数”。它把连续传播的组合逻辑切成离散步骤。上一个周期的结果在寄存器输出端稳定存在，成为本周期组合逻辑的输入；本周期算出的结果，要到下一个时钟沿才成为新的状态。

例如一个 8-bit 寄存器保存 **00101100**。只要写使能为 0，输入端怎样变化都不会改变 Q。写使能为 1 且时钟沿到来时，D 端的 8 个 bit 被一起采样。这里“一起”很重要：寄存器保存的是同一个边界上的一组 bit，不是一位一位随意变化。

二、计数器是带反馈的寄存器

计数器的下一值来自当前值加一：

```
next = current + 1
on clock edge:
    current = next
```

这个结构里有反馈，但反馈不是直接绕回去，而是经过加法器，并且被时钟边界切开。若 current 为 3，本周期组合逻辑算出 next 为 4；只有下一个时钟沿到来，寄存器才保存 4。随后组合逻辑再从 4 算出 5。

如果没有寄存器边界，current+1 的结果会立刻回到输入，再加一，再回到输入，电路无法表达“每个周期只增加一次”。计数器因此是理解同步电路的好例子：组合逻辑决定下一值，寄存器决定何时接受下一值。

计数器还会遇到宽度限制。4-bit 计数器从 **0000** 到 **1111**，再加一会回到 **0000**。这不是故障，而是硬件位宽决定的结果。后面讲溢出时会再次看到这个现象。

三、移位寄存器移动 bit

移位寄存器把一组 bit 向左或向右移动。它可以用于串行输入、串行输出、简单延迟线和位级变换。

```
next[3] = current[2]
next[2] = current[1]
next[1] = current[0]
next[0] = serial_in
```

假设 current 为 **1010**，serial_in 为 1。下一个时钟沿后，寄存器变成 **0101**。

注意，四个位不是按顺序一个一个搬。组合逻辑同时准备好四个 next 输入，时钟沿到来时一起写入。

这个例子说明状态更新不一定是加法，也可以是重新排列、选择、清零、保持或装载外部输入。寄存器提供状态位置，组合逻辑定义下一状态规则。

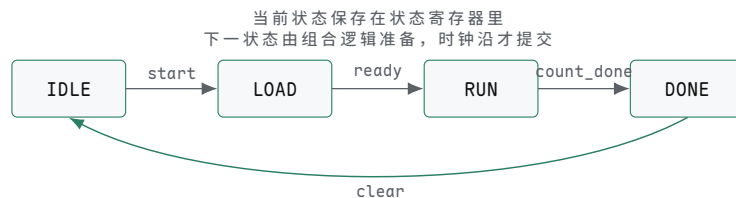
四、状态机把阶段保存下来

有限状态机由三部分组成：当前状态、输入条件、下一状态逻辑。当前状态保存在寄存器里，组合逻辑根据当前状态和输入条件决定下一状态。

以一个三阶段硬件控制器为例：

当前状态	条件	下一状态
IDLE	start=1	LOAD
LOAD	ready=1	RUN
RUN	count_done=1	DONE
DONE	clear=1	IDLE

若当前状态是 IDLE 且 start=0，状态保持 IDLE；若 start=1，下一状态逻辑准备 LOAD，等时钟沿到来后状态寄存器才真正变成 LOAD。状态机因此不会在一个周期里把 IDLE、LOAD、RUN、DONE 全部跑完，它每次只跨过一个明确边界。



图示：状态机每个周期只从当前状态跨到下一状态，不能在一个周期内把所有圆圈跑完。

状态机输出可以只由当前状态决定，也可以同时依赖输入。前者输出更稳定，后者反应更快但更容易受输入毛刺影响。这里先记住一条原则：状态机不是画几个圆圈，它是“状态寄存器 + 下一状态组合逻辑 + 输出组合逻辑”的硬件结构。

五、状态机为控制器铺路

现在硬件已经能够保存一组 bit、按周期加一、按周期移动、按条件切换阶段。下一步要把这些能力用于更大的动作组织：哪些寄存器在这个周期写入，ALU 做哪种运算，哪一路数据被选择，下一阶段去哪里。

这就是控制器的基础。控制器本质上也是组合逻辑和状态寄存器的配合，只是它输出的不是一个普通数据值，而是一组用来打开、关闭、选择其他硬件路径的控制信号。进入控制器之前，还需要先讲数的表示和算术电路。

本章的标注对象是“状态转移表”。只画状态圆圈不够，必须写清当前状态、输入条件、下一状态和输出。

- 纸上实验：为一个三状态控制器写表：IDLE 等待 start，RUN 计数，DONE 等待 clear。每一行都写出本周期输出和下一状态。
- 位级证据：寄存器是一组并排触发器；计数器是寄存器输出经过加法器反馈到输入；移位寄存器是固定连线选择了相邻 bit。
- 控制证据：状态机输出可以只依赖状态，也可以依赖状态和输入；后者反应快，但更容易把输入毛刺传给控制线。

- 检查题：若状态编码里出现未使用编码，电路上电或受干扰进入它时，下一状态逻辑应该怎样让机器回到安全状态？

经典系统教材会把状态机当作小型机器来读：状态寄存器是“当前 PC”，下一状态逻辑是“控制转移”，输出逻辑是“本周周期动作”。理解这点，后面控制器和 CPU 就不再神秘。

运算部件与数据通路

本部分导读

有了稳定状态和组合逻辑，就可以构造算术电路、ALU、标志位、寄存器阵列、内部连线和控制信号。这一部分仍然站在纯硬件角度：先让值能够被表示、计算、选择和保存，再谈怎样形成可重复动作。

第 7 章

数的表示与算术电路

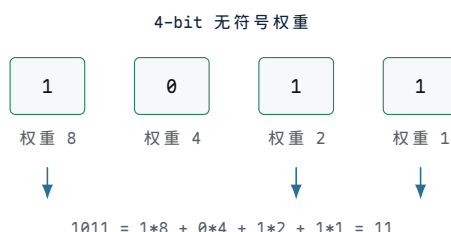
硬件只保存 bit。所谓整数、正负号、溢出，都是对 bit 模式的解释规则。若不先讲清楚表示，后面的加法器、减法器、比较器和 ALU 就会像在做抽象数学；事实上它们处理的始终是固定宽度的一组电线。

一、无符号数就是按权重求和

4-bit 无符号数 $b_3 b_2 b_1 b_0$ 的值是：

$$\text{value} = b_3 * 8 + b_2 * 4 + b_1 * 2 + b_0$$

因此 **1011** 表示 11，因为它等于 $8+0+2+1$ 。无符号数没有负数概念，所有 bit 都贡献非负权重。4-bit 能表达的范围是 0 到 15，8-bit 能表达的范围是 0 到 255。



图示：无符号数的每一位都有固定非负权重，硬件只保存 bit，数值来自解释规则。

位宽是硬件限制，不是显示格式。一个 4-bit 加法器只能输出 4 个结果 bit，再加上可能的进位。若 15 加 1，低 4 位回到 **0000**，最高位外产生一个进位。这个进位不是“可有可无的小细节”，它是无符号运算是否超出范围的证据。

二、补码让负数也能走加法器

硬件希望用同一个加法器处理加法和减法。补码表示提供了这个能力。以 4-bit 为例，最高位不再只是 8，而是解释成负权重 -8：

$$\text{value} = b_3 * (-8) + b_2 * 4 + b_1 * 2 + b_0$$

于是 **1111** 的值是 $-8+4+2+1=-1$ ，**1110** 的值是 $-8+4+2+0=-2$ ，**1000** 的值是 -8。4-bit 补码范围是 -8 到 7。

求一个数的相反数，可以按“取反加一”理解：

```
x = 0011 // 3
~x = 1100
~x + 1 = 1101 // -3 in 4-bit two's complement
```

为什么这个规则适合硬件？因为取反可以用一排 NOT 门完成，加一可以交给加法器。于是 $a-b$ 可以变成：

$$a - b = a + (\text{NOT } b) + 1$$

这样，减法器不必完全另做一套。只要在 b 输入前加反相控制，并把最低位进位输入设为 1，同一个加法器就能执行减法。

本章的核心标注是“同一组 bit，多种解释”。硬件只搬运和变换 bit，解释由编码规则和控制信号决定。

- 纸上实验：用 4-bit 同时解释 1000：无符号是 8，补码是 -8；再计算 $0111 + 0001$ ，分别判断无符号进位和有符号溢出。
- 位级证据：补码取负可写成按位取反再加一；减法可复用加法器执行 $a + (\sim b) + 1$ 。
- 标志证据：Carry 适合无符号边界，Overflow 适合补码有符号边界；不能把两者混成同一个“溢出”。
- 检查题： $1111 + 0001 = 0000$ 。它在无符号和补码解释下分别意味着什么？哪些标志应当被置位？

经典教材常把整数表示放在机器级程序前面讲，是因为后面的比较、分支、地址计算和溢出 bug 都来自这套位级解释。先学会看 bit，再谈“数”。

ALU 与标志位

ALU 是算术逻辑单元。它不是凭空出现的“大部件”，而是因为加法、减法、按位逻辑、移位和比较反复出现，硬件需要把这些能力集中到一个受控制的组合逻辑块中。ALU 的本质是：多个候选计算电路并排存在，控制信号选择本周期要看的结果。

一、ALU 的输入输出先从需求看

假设一块硬件已经能保存两个 8-bit 值 A 和 B。它可能需要 A+B，也可能需要 A-B，也可能只需要 A AND B 来屏蔽某些 bit，还可能需

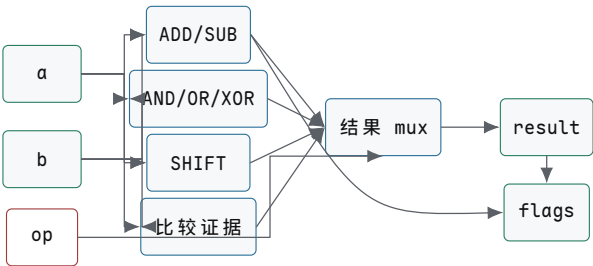
要判断 A 是否等于 B。若每个需求都单独拉一条完整路径，后面的连线和控制会很乱。

更自然的做法是给这组计算一个统一入口：

```
ALU(a, b, op) -> result, flags
```

a 和 b 是两个操作数，op 是选择信号，result 是结果，flags 是结果的附带证据。一个最小 ALU 可以包含：

op	运算	硬件来源
ADD	$a + b$	加法器
SUB	$a - b$	加法器加反相控制
AND	$a \& b$	按位 AND 门阵列
OR	$a b$	按位 OR 门阵列
XOR	$a \text{ xor } b$	按位 XOR 门阵列
SHIFT	移位	移位器



图示：ALU 把多种候选结果并排产生，再由 op 选择最终结果；标志位来自结果和关键内部证据。

二、减法复用加法器

ALU 里最值得细看的是 SUB。若单独做减法器，会增加面积和连线。补码让它复用 ADD 的加法器：

```
if op = ADD:
    b_to_adder = b
    carry_in = 0
if op = SUB:
    b_to_adder = NOT b
    carry_in = 1
result = a + b_to_adder + carry_in
```

这段伪代码描述的是硬件选择。实际电路里，b 的每一位前面可以接 XOR 门：当 sub 控制为 0，b 原样通过；当 sub 控制为 1，b 被取反。最低位进位输入也由 sub 控制。这样 ADD 和 SUB 共用同一条进位链。

这个案例很重要，因为它展示了硬件设计的常见思路：先找到数学规则之间的共用部分，再用少量控制信号复用同一块组合逻辑。

三、结果选择来自多路选择器

ALU 内部可以并行产生多个候选结果，再用多路选择器按 op 选出最终 result。

```
add_result
and_result
or_result
shift_result
result = mux(op, add_result, and_result, or_result, shift_result)
```

这不表示所有运算都“按顺序算一遍”。组合逻辑是并行传播的：加法器、按位逻辑、移位器都可能同时产生自己的候选输出。最终哪一路被后级看见，由 mux 决定。

代价也随之出现。候选电路越多，ALU 面积越大；mux 输入越多，选择延迟越大；若某个候选结果来自长进位链，它可能成为 ALU 的关键路径。因此 ALU 不是“把功能塞进去就完了”，还要考虑延迟、面积和功耗。

四、标志位是结果证据

ALU 常见标志位包括：

标志位	含义	用途
Zero	result 是否为 0	相等判断、条件选择
Carry	无符号进位或借位证据	多字长加法、无符号比较
Overflow	补码有符号溢出	有符号范围检查
Negative	result 最高位是否为 1	补码符号证据

Zero 可以由所有结果 bit 做 OR 后再取反得到。只要有任意一位为 1，结果就不是 0；所有位都是 0，Zero 才为 1。Negative 通常直接来自结果最高位。Carry 来自加法器最高位外的进位。Overflow 则要看有符号加减法中符号是否出现不可能的变化。

标志位不是额外装饰，而是后续控制的重要输入。若比较 A 和 B 是否相等，不一定要把完整结果拿出去再分析；ALU 做 A-B 后，Zero=1 就说明两者相等。

例如要判断两个 8-bit 值是否相等，可以让 ALU 执行减法。若结果 8 个 bit 全部为 0，Zero 置 1。控制器看到 Zero=1，就知道两个输入相等。这个判断没有另外发明一套“相等机器”，而是复用了减法路径和结果检测网络。

五、ALU 是组合部件，保存要靠寄存器

ALU 本身通常是组合逻辑。输入一变，结果经过传播延迟后改变。它不会记住上一次运算。若要保存 ALU 结果，必须在输出后接寄存器。

这个边界必须分清：ALU 负责计算，寄存器负责保存，控制信号负责选择。下一章会把 ALU 接到寄存器阵列、选择器和内部连线上，形成数据通路。没有数据通路，ALU 只是孤立计算块；没有 ALU，数据通路只能搬运，不能变换。

ALU 的经典读法是“候选计算并行产生，选择器选出一个结果，标志位保存结果证据”。不要把它想成一段顺序执行的程序。

- 纸上实验：给定 alu_op 四种取值 ADD、SUB、AND、XOR，写出每种情况下

结果、Zero、Negative、Carry、Overflow 的来源。

- 机器证据：减法复用加法器时，B 输入前的反相器和 carry-in=1 是控制信号驱动的真实硬件路径。
- 控制证据：分支不一定需要完整结果写回寄存器，控制器可能只读取 Zero 或 Negative 等标志来选择下一 PC。
- 检查题：若 ALU 结果已经正确，但 Zero 标志生成逻辑漏接最高位，哪类指令最先暴露问题？

经典教材会把 ALU 看成数据通路的“可配置组合核心”。它不是 CPU 全部，只是一个在控制线约束下把输入 bit 变成输出 bit 和证据 bit 的部件。

第 9 章

数据通路

数据通路回答一个问题：值从哪里来，经过哪里，最后保存到哪里。前面已经有了寄存器和 ALU，但孤立部件还不能完成连续动作。必须把寄存器阵列、选择器、ALU、内部连线 and 写回路接成一个可控制的闭环。

一、寄存器阵列提供多个状态位置

一个寄存器只能保存一个值。若硬件需要同时保存多个中间值，就需要寄存器阵列。寄存器阵列可以理解为一组编号寄存器，加上读写选择电路。

端口	作用	类型
读地址 A/B	选择两个源寄存器	输入
读数据 A/B	输出两个源值	输出
写地址	选择目标寄存器	输入
写数据	提供要保存的新值	输入
写使能	决定是否在时钟沿写入	输入

读地址进入译码和选择网络，决定哪两个寄存器的值出现在读数据端口。写地址进入译码器，写使能有效时，只允许被选中的那个寄存器在时钟沿接收写数据。这样，一组寄存器就能被编号访问，而不是每个寄存器单独拉一套控制线。

二、ALU 输入前必须选路

ALU 的两个输入不一定总来自同一处。一个输入可能来自寄存器阵列，也可能来自计数器或固定 0；另一个输入可能来自寄存器、常量或某个扩展后的字段。因此 ALU 输入前通常有 mux。

```
alu_a = mux(a_sel, reg_a, counter_value, zero)
alu_b = mux(b_sel, reg_b, constant, offset_value)
```

看一个具体动作：把寄存器 R1 和 R2 相加，结果写入 R3。数据通路需要让读地址 A 选择 R1，读地址 B 选择 R2；a_sel 选择 reg_a，b_sel 选择 reg_b；ALU op 选择 ADD。此时 ALU 输出就是 R1+R2。

若另一个动作是把某个计数器加 4，则 a_sel 选择 counter_value，b_sel 选择 constant，ALU op 仍然可以是 ADD。同一个 ALU 因为输入选择不同，服务了不同硬件动作。

三、写回路路径决定结果保存到哪里

ALU 输出不一定直接写回寄存器阵列。写回数据可能来自 ALU，也可能来自存储器读口、外部输入或计数器。写回前也需要 mux。

```
write_data = mux(wb_sel, alu_result, memory_data, input_data)
```

继续看 R1+R2 写入 R3 的动作。wb_sel 选择 alu_result，写地址选择 R3，reg_write 为 1。等时钟沿到来，R3 才真正保存结果。若 reg_write 为 0，即使 ALU 算出了结果，寄存器阵列也不会改变。

这就是“写回”的硬件含义：某一路结果被选择出来，在明确时钟边界写入某个状态位置。结果不是因为算出来就自动保存，必须有写地址、写数据、写使能和时钟边界同时配合。

四、内部连线不能被多个源同时强驱动

当多个部件都可能把值送到同一个目的地时，不能让它们同时强行驱动同一条线。要么用 mux 明确选择一路，要么用总线协议规定同一时刻只有一个驱动者。

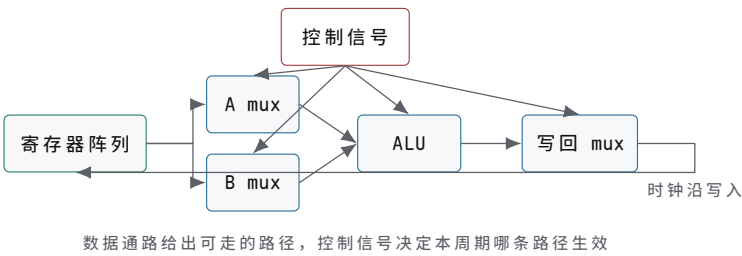
方案	做法	风险
mux 选择	多路输入收成一路输出	mux 层级太多会增加延迟
共享总线	多个源共享数据线	必须避免多个源同时驱动

例如 ALU 输出、存储器输出和外部输入都想进入写回端口。如果三者直接接在一起，一旦两个源同时输出不同电平，就会形成冲突。mux 让这件事变成明确选择：当前周期只允许一路成为 write_data。

五、数据通路形成可计算闭环

一个最小数据通路闭环可以写成：

```
register file -> mux -> ALU -> mux -> register file
```



图示：最小数据通路是读旧状态、选输入、计算、选写回、在边界保存新状态的闭环。它能在一个周期内读取旧值、选择输入、完成组合计算，并在时钟沿保存结果。这个闭环已经能表达很多动作，但它还不会自己决定“当前周期做什么”。哪些地址送入寄存器阵列，ALU 做加法还是减法，写回选择哪一路，写使能是否打开，都需要另一个硬件部件统一产生。下一章就是控制器。

数据通路的标注方式是 trace：从哪个状态元件读出旧值，经过哪些 mux 和组合部件，到哪个状态元件写入新值。

- 纸上实验：为 $R3 = R1 + R2$ 写一行 trace：读地址、ALU 输入选择、alu_op、写回选择、写地址、写使能。
- 机器证据：寄存器读口通常是组合输出，写口通常在时钟沿提交；读旧值和写新值不能混成同一个瞬间。
- 冲突证据：内部总线或共享线同一时刻只能由一个源驱动；多源并联必须通过 mux、三态规则或仲裁结构解决。
- 检查题：若写回 mux 选择线接错，ALU 算对了但寄存器结果错。排查时应看 ALU 输出，还是写回路径和写使能？

经典教材常把每条机器指令拆成数据通路上的值流。这里还没有完整指令集，但同样的方法已经可用：看值从哪里来，到哪里去，哪条控制线让它走这条路。

控制器

数据通路提供了很多可能路径，但同一周期不能所有路径都随意打开。控制器的任务是根据当前状态和外部条件，产生一组控制信号，让数据通路执行一个明确动作。控制器不是写在纸上的命令列表，而是一块输出控制电线的硬件。

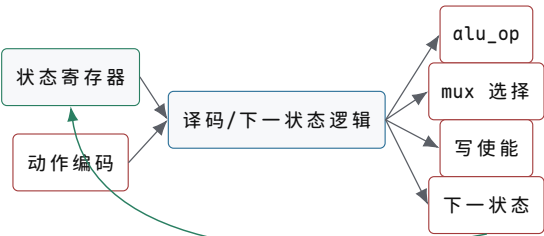
一、控制信号就是电线值

控制信号通常是一组 0/1 或多 bit 选择值：

```
alu_op
a_sel
b_sel
wb_sel
reg_write
mem_read
mem_write
next_state_sel
```

这些信号直接接到 ALU、mux、寄存器阵列和存储器接口。控制信号给错，数据通路就会走错。例如 reg_write 本应为 0 却变成 1，某个寄存器会在时钟沿被意外覆盖；wb_sel 本应选择 ALU，却选了外部输入，写回值就会错。

因此控制器要解决的不是“告诉硬件一句话”，而是“在每个周期给出一组一致的电线值”。一致意味着：来源只开一路，目的地只写该写的，ALU 操作和输入选择匹配，下一状态也和当前动作匹配。



控制器输出的是一组同时有效的电线值，不是一句顺序命令

图示：控制器把当前状态和动作编码译成一致的控制信号组，并准备下一状态。

二、硬连线控制适合简单规则

若动作规则简单，可以用组合逻辑直接从输入生成控制信号。例如输入 mode 为 0 时做加法，为 1 时做按位 AND：

```
mode = 0 -> alu_op = ADD
mode = 1 -> alu_op = AND
```

硬件实现仍然是译码和门电路，不是顺序执行。mode 进入译码逻辑，译码逻辑输出 ALU 的选择位。若还要控制写回，则同一块译码逻辑同时产生 wb_sel 和 reg_write。

硬连线控制速度快，结构也直接。缺点是规则复杂后，组合逻辑会变大，修改一个动作可能影响多条控制线。即便如此，很多简单数据通路仍然适合硬连线控制，因为它没有额外状态，输入一稳定，控制信号就稳定。

三、状态机控制多周期动作

如果一个动作无法在一个周期内完成，就需要状态机。状态机保存“当前走到哪一步”，每个状态输出一组控制信号，并决定下一状态。

状态	控制动作	下一状态
S0	选择源寄存器，锁存操作数	S1
S1	ALU 计算，结果进入临时寄存器	S2
S2	选择临时结果，写回目标寄存器	S0

这里每一行都对应真实硬件动作。S0 打开读路径和临时寄存器写使能；S1 设置 ALU 输入和 op，并保存 ALU 结果；S2 打开写回路路径和目标寄存器写使能。状态机让复杂动作分成多个清楚边界，而不是把所有组合逻辑塞进一个超长周期。

四、微操作把动作拆成控制信号组

复杂动作可以拆成微操作：读、选择、计算、保存、更新状态。这里的微操作不是另一个抽象层的口号，而是一个周期内同时打开的一组硬件控制信号。用一个具体动作看得最清楚：把 R1 和 R2 相加，结果写入 R3。

先假设数据通路里有这些部件：寄存器阵列有两个读地址输入和一个写地址输入；读出的两个值可以进入 A、B 两个操作数锁存器；ALU 的输出可以进入结果锁存器；写回 mux 可以在不同来源中选择一个值送回寄存器阵列。控制器要做的事，就是在每个状态给这些部件分配正确的选择值和写使能。下面先给出三个周期的控制信号 trace，再逐步解释每一组信号对应的硬件动作：

```

S0: read_sel_a=R1, read_sel_b=R2
    A_write=1, B_write=1
    reg_write=0, ALU_out_write=0

S1: a_sel=A, b_sel=B, alu_op=ADD
    ALU_out_write=1
    reg_write=0, A_write=0, B_write=0

S2: wb_sel=ALU_out, write_sel=R3
    reg_write=1
    A_write=0, B_write=0, ALU_out_write=0

```

第一步 S0 不是“把 R1 和 R2 读出来”这么一句话。read_sel_a 选中 R1 后，寄存器阵列的第一个读端口会把 R1 当前保存的 bit 组合驱动到读数据线上；read_sel_b 选中 R2 后，第二个读端口同理输出 R2。等这些组合路径稳定后，时钟沿到来，A_write 和 B_write 让 A、B 锁存器保存这两个值。这个周期里 reg_write 必须为 0，因为此时写回 mux 上的值并不代表最终结果，提前写寄存器只会制造错误状态。

第二步 S1 才让 ALU 工作。A、B 锁存器的输出作为 ALU 输入，alu_op=ADD 让 ALU 内部选择加法路径。加法电路不是瞬间得到稳定结果，低位进位会影响高位，输出线上可能经历一小段变化过程。控制器在这个周期打开 ALU_out_write，让时钟沿把已经稳定的 ALU 输出锁存下来；同时 reg_write 仍然保持 0，避免把尚未明确提交边界的值写进 R3。

第三步 S2 才写回。wb_sel 选择 ALU_out 锁存器，write_sel 选择 R3，reg_write 在这个周期置 1。到时钟沿时，寄存器阵列把写数据端口上的值保存到 R3。至此，R1 和 R2 没有被改动，R3 得到 R1+R2 的结果，控制器回到空闲或取下一动作的状态。

这个例子也说明控制错误为什么危险。若 S2 的 wb_sel 错选成 B 锁存器，R3 写入的就是 R2，而不是相加结果；若 S1 提前打开 reg_write，R3 可能在 ALU 输出尚未稳定时被覆盖；若 S0 忘记关闭 ALU_out_write，旧的计算边界会被无关读

数污染。微操作的价值就在这里：每一步都能检查“哪些线打开、哪些线关闭、哪个状态元件会在时钟沿改变”。

五、控制器走向最小 CPU

到这里，控制器已经具备两种能力：根据编码生成控制信号，根据状态推进多周期动作。下一章把“动作编码”固定存放在一个可读取的位置，把“当前读取到哪一条编码”保存在一个计数寄存器里，再让控制器反复取出编码、译码、驱动数据通路。这样，一个最小 CPU 就会自然出现。

控制器的标注对象不是单个输出，而是一组控制信号的时间表。每个周期哪些写使能为 1，哪些 mux 选择哪一路，ALU 做什么，都要能列成 trace。

- 纸上实验：为多周期 `ADD R3,R1,R2` 写 S0/S1/S2 三行控制表，检查每行是否只提交应该提交的状态。
- 机器证据：控制信号也是电线值；错误控制线会打开错误路径，比 ALU 算错更隐蔽。
- 状态证据：硬连线控制适合简单组合译码；多周期控制需要状态寄存器保存当前阶段。
- 检查题：若 S1 周期提前打开 `reg_write`，为什么 R3 可能写入不稳定值？这个错误属于数据通路，还是控制器时序？

经典教材会把控制器和数据通路并排讲：数据通路说明“能走哪些路”，控制器说明“本周期允许哪条路”。两者合起来，才是可执行动作。

最小 CPU 与整机硬件

本部分导读

前面已经有了计算、状态、选择和控制。现在把它们合成一个最小 CPU，再继续向外连接主存、总线、设备、中断控制器和启动硬件。

最小 CPU

现在才正式引入 CPU。前面已经有了逻辑门、组合逻辑、触发器、时钟、寄存器、ALU、数据通路和控制器。CPU 可以先理解成：用控制器驱动数据通路，按固定编码重复取出动作描述并执行的同步硬件。

一、为什么还需要动作编码

只有数据通路和控制器时，硬件能完成某些固定动作。例如控制器可以永远让 R1+R2 写回 R3。但若希望同一套数据通路在不同时间做不同动作，就需要把“这一次做什么”也表示成 bit。

动作编码就是把控制意图压成一串字段。字段不需要一开始想得很复杂，最小形式可以是：

```
opcode | dst | src_a | src_b
```

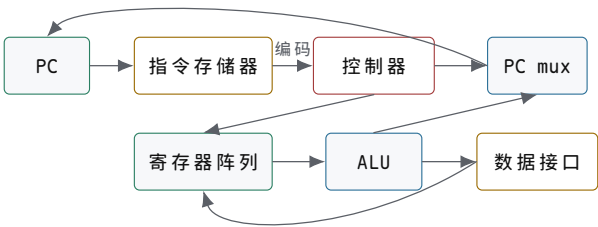
opcode 表示动作类型，例如 ADD、SUB、AND；dst 表示写入哪个寄存器；src_a 和 src_b 表示读取哪两个寄存器。控制器读取这些字段后，把它们译成 ALU op、读地址、写地址、写使能和写回选择。

这样，控制器不再只做一组固定动作，而是根据当前编码产生不同控制信号。编码本身仍然是硬件输入，不是文字命令。

二、PC 保存下一条编码的位置

若有一组动作编码连续放在存储结构中，硬件还需要知道当前取哪一条。这个位置由 PC 保存。PC 本质上也是寄存器，只是它保存的是“下一条动作编码的地址”。

部件	作用
PC	保存下一条编码地址
指令存储器	根据 PC 输出当前编码
控制器	根据编码字段产生控制信号
寄存器阵列	保存通用数据
ALU	做整数和逻辑运算
选择器网络	选择 ALU 输入、写回来源和下一 PC



时钟沿提交：写寄存器、更新 PC，下一周期再取下一条编码

图示：最小 CPU 是 PC、编码存储、控制器、寄存器阵列、ALU 和写回/下一 PC 选择组成的同步闭环。

顺序执行时，下一 PC 可以由加法器产生：

```
pc_next = pc_current + 4
```

这里的 4 不是神秘数字，只表示每条编码占 4 个字节时，下一条编码的地址比当前地址大 4。若编码宽度不同，推进量也会不同。

三、取指、译码、执行、写回是一条硬件路径

最小单周期模型可以这样走：

阶段	硬件动作	边界
取指	PC 送入指令存储器，得到编码	组合路径
译码	控制器产生 mux、ALU、写使能信号	组合路径
执行	寄存器阵列读值，ALU 计算	组合路径
写回	结果在时钟沿写入寄存器或 PC	状态边界

这四个名字容易让人误以为 CPU 在纸面上按语句执行。对硬件来说，它们是一条从状态出发、经过组合逻辑、回到状态的路径：PC 是状态，寄存器阵列是状态；指令存储器、译码器、ALU 和 mux 是组合逻辑；写回和 PC 更新发生在时钟沿。

用一条普通运算编码走一遍：当前位置 $PC=100$ ，指令存储器在地址 100 处保存的是

```
ADD dst=R3, src_a=R1, src_b=R2
```

周期刚开始时，PC 寄存器的输出稳定为 100。这个地址进入指令存储器，存储器经过地址译码后把这条编码的 bit 输出出来。控制器看到 opcode=ADD，于是把 alu_op 设为 ADD，把寄存器阵列的两个读地址分别设为 R1 和 R2，把写地址设为 R3，把 wb_sel 设为 ALU，把 reg_write 设为 1，同时让 next_pc 选择 PC+4。

这些控制信号稳定后，寄存器阵列的两个读端口输出 R1 和 R2 当前的值。它们经过 ALU 输入 mux 进入 ALU，加法路径产生结果。结果再经过写回 mux，到达寄存器阵列的写数据端口。另一边，PC+4 加法器把 100 和 4 相加，PC mux 选出 104，送到 PC 的下一值输入。

注意，这一长串变化都发生在同一个时钟周期的组合路径里。R3 不会在中途多次保存临时值，PC 也不会组合逻辑刚出现 104 时立刻改变。只有到时钟沿，两个状态变化才同时提交：R3 保存 ALU 结果，PC 保存 104。下一个周期开始时，CPU 才会从地址 104 取下一条编码。

这个走读能暴露单周期 CPU 的代价。一个周期必须容纳 PC 输出、指令存储器访问、控制译码、寄存器读取、ALU 计算、写回 mux、寄存器建立时间，以及 PC+4 和 PC mux 的路径。若其中任何一段变慢，时钟周期都要被迫拉长。多周期 CPU 或流水线 CPU 后来要解决的，就是把这条过长路径切成多个边界；但最小 CPU 先保留这条完整路径，便于看清所有部件怎样闭合。

四、分支只是选择下一 PC

顺序情况下，下一 PC 通常是 PC+4。若需要跳到另一处编码，改变的只是 PC 的下一值来源。条件分支可以由 ALU 比较结果和选择器完成：

```
branch_taken = condition_met AND branch_op
next_pc = mux(branch_taken, pc_plus_4, branch_target)
```

例如一条“若 R1 等于 R2 就跳转”的编码。数据通路读取 R1 和 R2，ALU 做减法或相等检测，Zero 标志表示两者是否相等。控制器看到这是分支类 opcode，就把 branch_op 置为 1。若 Zero 也为 1，next_pc 选择 branch_target；否则选择 pc_plus_4。

把它拆开看。假设 $PC=200$ ，编码是：

```
BEQ src_a=R1, src_b=R2, target=320
```

PC=200 先取出这条编码。控制器从 opcode 看出它不写普通寄存器，于是 `reg_write=0`；从 src 字段选出 R1 和 R2，让寄存器阵列输出两个值；再让 ALU 做相等检测，或者做 `R1-R2` 并观察结果是否为 0。若两个值相等，`Zero=1`；若不相等，`Zero=0`。与此同时，PC+4 路径算出 204，目标地址路径准备好 320。

真正决定下一条从哪里取的，是 PC 前面的 mux。这个 mux 至少有两个输入：顺序地址 PC+4 和分支目标 `branch_target`。选择信号可以写成：

```
take_branch = is_branch AND Zero
```

当 `is_branch=1` 且 `Zero=1` 时，PC mux 输出 320；当 `is_branch=1` 但 `Zero=0` 时，PC mux 输出 204。到时钟沿，PC 只会保存其中一个值。也就是说，分支并没有让 CPU “跳过一段文字”，它只是改变了 PC 这个寄存器的下一值。下一周期取到哪条编码，完全由这个新 PC 决定。

分支控制最容易出错的地方也在这里。若 `reg_write` 没关，分支周期可能误写某个通用寄存器；若 `Zero` 的含义和控制器约定相反，相等时反而不跳；若 PC mux 的选择线接反，顺序地址和目标地址会互换。把分支理解成“下一 PC 选择问题”，这些错误就能逐一落到具体电线和 mux 上检查。

五、最小 CPU 的边界

这台 CPU 还没有流水线、cache、异常、权限、复杂互连和真实设备。但它已经把本书前十章合成了一个闭环：编码选择控制信号，控制信号驱动数据通路，数据通路计算和保存新状态，PC 推进到下一条编码。

最小 CPU 的价值不在于性能，而在于把硬件关系说明白。寄存器保存状态，ALU 计算，mux 选路，控制器给出控制线，PC 决定下一条编码位置，时钟沿统一提交状态变化。下一章把它接向主存、总线和设备，形成整机硬件。

最小 CPU 的经典标注是“状态、组合路径、提交边界”三件套。每条指令都要能拆成 PC 旧值、编码、控制线、数据通路值流和时钟沿提交。

- 纸上实验：走读 `ADD R3,R1,R2` 和 `BEQ R1,R2,target`，分别写出 PC 下一值、`reg_write`、ALU 操作、写回目标。
- 状态证据：PC 和寄存器阵列是状态；指令存储器、译码器、ALU、mux 是组合路径；写回发生在边界。
- 分支证据：分支不是“跳过文字”，而是 PC mux 选择了非顺序下一值。
- 检查题：若一条分支指令误写了通用寄存器，应该检查 opcode 译码、`reg_write` 默认值，还是写地址字段？三者都可能。

经典教材讲机器级执行时，常把一条指令解释成硬件中的一条值流。本章已经具备这种读法：编码不是抽象命令，而是一组会驱动控制线 and 数据路径的 bit。

内存、总线、I/O 与整机

最小 CPU 能执行编码，但它还不是整机。整机需要更大的存储空间、部件间通信路径、设备控制方式、异步事件入口和复位启动路径。本章只站在硬件角度收束这些结构：数据怎样离开 CPU，怎样到达主存或设备，结果又怎样回来。

一、主存扩展状态空间

寄存器阵列很快，但容量有限。若所有状态都放在寄存器里，编号、连线、读写端口都会迅速膨胀。主存提供大容量状态空间，用地址选择位置，用数据线传递内容，用控制信号说明读还是写。

信号	作用
地址	指出访问哪个存储位置
写数据	写入时提供数据
读数据	读取时返回数据
读写控制	指明本次访问方向
ready/valid	指明请求或响应是否有效

一次读访问可以这样理解：CPU 给出地址和读控制，主存经过内部译码、阵列访问和输出驱动后，把数据放到读数据线上，并用响应信号表示结果有效。一次写访问则是 CPU 同时给出地址、写数据和写控制，主存在合适边界把数据保存到对应位置。

主存和寄存器的差别不只是容量。寄存器通常直接接在 CPU 数据通路里，延迟短、端口少而固定；主存容量大，但访问路径更长，需要地址译码、阵列选择、总线传输和握手。

二、快慢差距逼出层级存储

CPU 内部一个周期可以很短，而主存访问可能慢得多。若每次取编码、读数据、写数据都等主存完成，ALU、寄存器阵列和控制器会长时间空转。层级存储的动机，就是把最近、最常用的一小部分内容放到更近、更快的位置。

cache 可以先理解成靠近 CPU 的小型硬件存储。它不是一句“先临时放一下”的比喻，而是一组硬件表项：

字段	作用
tag	判断当前地址是否对应这一项
data	保存从主存取来的数据块
valid	表示这一项是否有效
dirty	表示这一项是否被改过、尚未写回下层

命中时，数据从 cache 返回，CPU 不必等主存。未命中时，硬件要向下层请求数据块，再填入 cache。若被替换的块是 dirty，还要先写回。这里先抓住硬件链路：cache 解决的是速度差距，但它引入了命中判断、替换、写回和一致性边界。

用一次读数据看这条链路。CPU 内部的数据通路算出地址 A，并发出读请求。地址 A 先被拆成几部分：低位选择块内的哪几个字节，中间位选择 cache 的某一组，高位作为 tag 和表项里保存的 tag 比较。若这一组中存在 valid=1 且 tag 相等的表项，就是命中。此时数据阵列读出整块数据，再由块内偏移选出需要的字节或字，送回 CPU 的读数据路径。对 CPU 来说，这次访问很快结束；对硬件来说，实

际发生的是索引、tag 比较、有效位判断、数据选择和响应返回。

未命中时路径明显更长。cache 发现没有匹配表项后，不能凭空产生数据，只能向下层主存发出读块请求。若准备替换的旧表项 dirty=0，可以直接覆盖；若 dirty=1，说明旧块被改过但下层还没有最新内容，cache 必须先把旧块地址和旧数据写回主存，等写回完成后再读取新块。新块返回后，cache 更新 data、tag、valid，必要时清除 dirty，然后把 CPU 本来要的那一部分数据返回。这个过程解释了为什么同样是读一个地址，命中和未命中的硬件代价完全不同。

写访问还要再区分。若写命中，cache 可以直接修改对应字节，并把 dirty 置 1，表示这份内容已经比下层新；也可以同时写下层，保持下层同步。若写未命中，硬件策略会决定是先把整块取入 cache 再改，还是绕过 cache 直接写下层。不同策略会影响性能和复杂度，但核心问题一样：每一层都必须清楚“哪个副本是最新的、什么时候需要写回、CPU 何时可以认为写入完成”。



层级存储用更近的小容量硬件吸收大量近期访问，未命中才向下层请求

图示：层级存储不是单个部件，而是用速度、容量和距离形成的硬件链路。

三、总线和互连连接多个目标

当 CPU、cache、主存、设备控制器都要通信时，需要互连结构。简单系统可以用共享总线，复杂系统会用分层互连或片上网络。无论形式如何，它至少要解决五个问题：

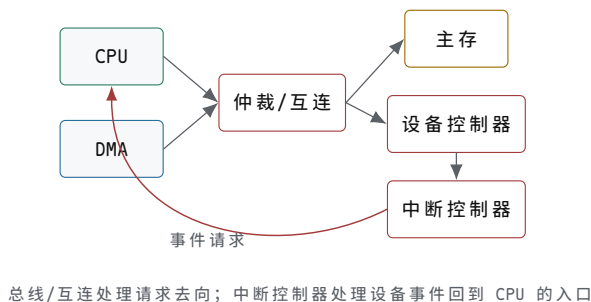
- 谁发起请求。
- 地址指向哪个目标。
- 数据往哪边走。
- 多个发起者冲突时谁先走。
- 响应怎样回到发起者。

共享总线的直觉是：多个部件共用一组地址线、数据线和控制线。由于同一时刻不能让多个源同时驱动总线，就需要仲裁器决定谁获得使用权。分层互连则把不同目标挂在不同分支上，通过译码和路由把请求送到正确目标。

互连不是“几根线”这么简单。它要保证电气上不冲突，时序上能握手，地址上能译码，响应上能返回正确发起者。整机复杂度很大一部分来自这里。

看一个最简单的互连访问。CPU 发出一个读请求，地址落在主存范围。互连先接收请求信号、地址和发起者编号，然后地址译码逻辑判断这个地址应该送到主存控制器，而不是设备控制器。若此时总线空闲，仲裁器授权 CPU，地址和控制信号被送到主存侧；若 DMA 正在占用总线，CPU 的请求可能要等待。主存返回数据时，互连还要把响应送回原来的发起者。若系统里只有 CPU 一个发起者，这件事看起来很简单；一旦 DMA、图形设备或其他主设备也能发请求，响应通道就必须带着足够的信息，保证数据不会回错对象。

再看地址落在设备范围的情况。CPU 发出的仍然是“地址、读写方向、数据、有效信号”这类硬件请求，但互连译码后不会把它送到主存，而是送到某个设备控制器。设备控制器看到的是自己的寄存器地址：可能是状态寄存器，可能是命令寄存器，也可能是数据缓冲寄存器。CPU 访问主存和访问设备，在数据通路入口处形式很像；真正的分岔发生在互连的地址译码处。



图示：整机互连同时要处理访问请求、设备寄存器、DMA 搬运和中断事件。

四、设备通过寄存器、DMA 和中断接入

设备控制器通常把自己的控制寄存器映射到地址空间中。CPU 对某些地址读写，实际访问的是设备寄存器，这就是 MMIO。写一个控制寄存器可能启动设备动作，读一个状态寄存器可能得到设备是否完成、是否出错、缓冲区是否可用。

若设备要搬运大量数据，让 CPU 一字一字读写会浪费内部数据通路时间。DMA 控制器可以在获得总线使用权后，直接在设备和主存之间搬运数据。CPU 只需要配置源地址、目标地址、长度和方向，真正的数据搬运由 DMA 硬件完成。

设备完成工作或状态变化时，需要把事件送回 CPU。中断控制器负责收集多个设备的请求，进行屏蔽、优先级排序和投递。站在本书角度，中断先看成硬件事件入口：外部设备把一条请求线送进控制结构，CPU 在规定边界看到这个请求，并转入对应处理路径。

先看一次 MMIO 写。假设某个设备控制器有一个命令寄存器，地址是 D。CPU 执行一次写访问，地址线给出 D，写数据线上放入命令 bit，写控制有效。互连把 D 译码到这个设备控制器。设备控制器内部不是普通主存阵列，而是一组控制寄存器和状态机；它在握手边界接收写入值，把命令寄存器更新为新值，并可能让内部状态机从 idle 进入 busy。若命令要求设备开始工作，真正的机械、电气或外部协议动作会在设备控制器内部继续推进。CPU 写的是一个地址，硬件实际改变的是设备控制器的状态。

再看一次 MMIO 读。CPU 读取设备状态地址 S，互连把请求送到设备控制器。控制器把当前状态寄存器的值放到读数据线上，例如 busy 位、done 位、error 位、buffer_ready 位。这个读操作不一定只是“看一眼”。有些设备寄存器会在读取后清除某些事件位，有些寄存器要求按特定顺序访问，否则会读到旧状态或破坏握手。因此设备寄存器虽然映射在地址空间里，语义却由设备控制器硬件定义。

DMA 的链路更能体现整机配合。若不用 DMA，CPU 要把一大块设备数据搬到主存，通常需要反复执行“读设备数据寄存器、写主存地址、地址递增、计数递减”这样的动作。每搬一个字，CPU 的寄存器、ALU、互连和设备寄存器都要参与一次。若用 DMA，CPU 先通过 MMIO 配置 DMA 控制器：源端是设备数据端口，目标是主存起始地址，长度是 N，方向是设备到主存，然后写启动位。之后 DMA 控制器自己成为总线发起者：它向设备侧取数据，向主存侧写数据，内部地址计数器递增，长度计数器递减，直到 N 变成 0。CPU 数据通路不再逐字搬运，但互连、主存控制器和设备控制器仍然在硬件层面真实传输每一个数据块。

DMA 也带来新的硬件边界。因为 CPU 和 DMA 都可能访问主存，互连必须仲裁；因为 CPU 可能从 cache 里看见旧数据，系统必须规定 DMA 写入的主存区域怎样和 cache 保持一致；因为设备可能比主存慢，DMA 控制器还要处理等待、重试或分批传输。这里不需要先引入操作系统，只要看到硬件事实：一旦有多个发起者同时读写共享存储，仲裁、可见性和完成信号就变成整机必须解决的问题。

最后看中断。设备完成 DMA 后，可以把 done 位写进自己的状态寄存器，同时拉起一条中断请求线。中断控制器接收来自多个设备的请求，先检查屏蔽位，再按优先级选择一个请求，把它整理成 CPU 能看到的事件信号。CPU 不会在任意组合逻辑中间突然改变动作，它通常在指令边界或规定的状态边界采样这个事件。采样到

有效中断后，控制结构选择一条入口路径：保存必要状态，把 PC 改到事件入口地址，后续再由那里的编码读取设备状态并处理结果。

这条链路可以和轮询对比。轮询时，CPU 反复通过 MMIO 读取设备状态寄存器，直到 done 位变成 1；中断时，CPU 不必一直读状态寄存器，而是由设备完成后主动通过中断控制器发出事件。两者都建立在同一套硬件事实之上：设备状态保存在设备控制器里，CPU 通过互连读写这些寄存器，中断只是额外提供了一条“设备事件回到 CPU 控制入口”的硬件通道。

五、复位启动给整机第一步

通电或复位后，CPU、主存控制器、互连和设备控制器都必须进入规定初始状态。PC 通常被置到固定启动地址，那里连接 ROM、片上存储或其他启动介质。启动路径的第一件事，是让硬件从可预测位置开始取编码。

整机启动不仅是 CPU 复位。时钟源要稳定，复位树要按顺序释放，主存控制器要完成训练或初始化，互连要进入可转发请求的状态，必要设备要处于安全默认值。若这些硬件条件没有满足，即使 CPU 本身能取指，也可能在第一次访问主存或设备时卡住。

到这里，本书的硬件链路闭合：电和器件形成可控开关，开关形成门，门形成组合逻辑，反馈和时钟形成状态，寄存器和 ALU 形成数据通路，控制器组织动作，最小 CPU 执行编码，主存、cache、互连、设备和启动路径把它扩展成整机。

整机硬件的标注对象是事务。一次访问不是“读内存”三个字，而是地址、方向、数据、发起者、目标、握手、响应和错误状态组成的一次硬件事务。

- 纸上实验：分别走读一次 cache 命中读、cache 未命中读、MMIO 写设备命令、DMA 完成并发中断，标出每一步的发起者和响应者。
- 机器证据：主存、cache、设备寄存器都可以被地址访问，但语义不同；地址译码决定请求落到哪类目标。
- 并发证据：DMA 和 CPU 都能成为总线发起者，必须有仲裁；cache 和 DMA 共享主存时，必须有一致性或同步规则。
- 检查题：CPU 写设备命令寄存器后，能否立刻假设设备动作完成？应该看 MMIO 写完成、设备 done 位、中断，还是 DMA completion？

经典系统教材会把整机看成多层抽象：指令看到地址，硬件看到事务，设备看到寄存器和状态机，软件最终看到完成事件。能把这几层对应起来，才算从最小 CPU 走到了真正机器。

读完后的检查表

读完本书后，至少应该能回答下面这些问题：

主题	能力要求
比特	能说明为什么计算机用两个稳定区间表示 0 和 1，而不是追求连续电压的精确值。
逻辑门	能从开关路径解释 NOT、AND、OR、NAND 的行为，并理解为什么 NAND 可以构造其他门。
组合逻辑	能把真值表转成门电路，解释半加器、全加器、多路选择器和译码器分别解决什么问题。
时序逻辑	能说明锁存器、触发器、寄存器和时钟边界的关系，知道为什么组合逻辑不能保存历史。
状态机	能把状态、输入、转移条件和输出分开，并说明状态机怎样控制一组硬件动作。
ALU	能把加法、减法、按位运算、比较和标志位放在同一个计算部件里解释。
最小 CPU	能画出 PC、指令存储器、寄存器堆、ALU、数据存储器 and 控制器之间的数据流。
整机硬件	能说明主存、总线、MMIO、DMA、中断控制器和复位启动怎样把最小 CPU 扩展成整机。

最终检查不以“能不能复述章节标题”为准，而以能不能给出机器证据为准。

- 给一个信号节点，能说出它何时被拉高、何时被拉低、何时悬空、何时冲突。
- 给一个组合电路，能列真值表、最长路径、可能毛刺和后级采样边界。
- 给一个状态电路，能指出旧状态、下一状态、时钟沿、建立/保持约束和复位行为。
- 给一个算术结果，能分别按无符号和补码解释，并说明 Carry 与 Overflow 的差别。
- 给一条最小 CPU 指令，能写出控制信号 trace、数据通路 trace 和 PC 下一值。
- 给一次内存或设备访问，能写出地址译码、总线/互连事务、响应返回和完成证据。

如果这些证据能写出来，说明读者已经把“电平 -> 门 -> 组合逻辑 -> 状态 -> 数据通路 -> 控制器 -> CPU -> 整机”的链路连成了一台机器。